

JUGEND FORSCHT 2027

Regionalwettbewerb Nordrhein-Westfalen (Ruhrgebiet)

Fachgebiet: Technik

MIRA

Machine Intelligence for Recycling Automation

Edge-KI-basierte Erkennung und mechatronische Sortierung von Sekundärrohstoffen

Jungforscher:	Jeremy Darko
Jahrgangsstufe:	Stufe EF (Jahrgangsstufe 11)
Schule:	Gymnasium Broich
Schulort:	Mülheim an der Ruhr
Datum der Einreichung:	15. Januar 2027

Projektüberblick

Das Projekt MIRA (Machine Intelligence for Recycling Automation) untersucht, wie moderne Deep-Learning-Verfahren zur dezentralen Wertstoffsartierung auf ressourcenbeschränkter Edge-Hardware eingesetzt werden können. Im Rahmen einer sechsstufigen experimentellen Untersuchung (EXP-001 bis EXP-015) wurde ein System entwickelt, das von einer einfachen Bildklassifizierung mit 4 Klassen (MobileNetV2) zu einer komplexen Multi-Objekt-Erkennung mit 5 Klassen (YOLO11n) skaliert wurde [sandler2018mobilenetv2, ultralyticsyolov8]. Durch Post-Training-Quantisierung konnte das Modell um den Faktor 9 auf 2.61 MB (Klassifikation) beziehungsweise 2.90 MB (Detektion, YOLO11n) komprimiert werden [deng2020model]. Mit einer CPU-Inferenzlatenz von nur 10.32 ms (EXP-004) beziehungsweise 46.0 ms (EXP-009) und einer mAP50 von 60.7 % (EXP-014, bestes Detektionsmodell) beweist die Arbeit die Praxistauglichkeit kostengünstiger Sortiersysteme im Echtzeitbetrieb.

Zusammenfassung

Die manuelle Sortierung von Recyclingmaterialien ist zeitaufwändig, kostenintensiv und fehleranfällig. Diese Arbeit untersucht im Rahmen des Projekts *MIRA*, wie verschiedene tiefe neuronale Netzwerkarchitekturen die automatische Klassifikation und räumliche Lokalisierung von Wertstoffen auf ressourcenbeschränkten Edge-Geräten beeinflussen. Dabei wurde in Stage A (Klassifikation) ein vierklassiges System (Glas, Metall, Papier, Plastik) als Machbarkeitsnachweis etabliert, welches in Stage B (Detektion) um eine fünfte Klasse (Restmüll) für praxisnahe Abfallströme erweitert wurde.

Durch einen systematischen Vergleich eines von Grund auf trainierten Convolutional Neural Networks (Scratch-CNN), eines vortrainierten MobileNetV2 mit Transfer Learning (Frozen Base) und eines verfeinerten MobileNetV2 (Fine-Tuning) wurden Klassifikationsgenauigkeit, Inferenzgeschwindigkeit und Modellgröße evaluiert [yosinski2014transferable]. Das fine-tuned MobileNetV2 erreichte eine Validierungsgenauigkeit von 87.42 % und übertraf das Scratch-CNN um 26.42 Prozentpunkte. Zur Optimierung für Edge-Hardware wurde das Keras-Modell mittels Post-Training-Quantisierung auf 8-Bit-Integers (INT8) konvertiert [deng2020model]. Dies reduzierte die Modellgröße um das 9-Fache auf 2.61 MB bei einer durchschnittlichen CPU-Inferenzzeit von nur 10.32 ms ohne Genauigkeitsverlust.

Um reale Multi-Objekt-Szenarien abzubilden, wurde das System auf YOLO11n mit 5 Klassen migriert [ultralyticsyolov8]. Ein systematischer Datensatzvergleich (EXP-013 bis EXP-015) über TACO [proenca2020taco], Stanford TrashNet [yang2016classification] und Roboflow ergab, dass die Kombination TACO+TrashNet+Roboflow (EXP-014) mit 60.7 % mAP50 das beste generalisierbare Detektionsergebnis liefert. Durch INT8-Quantisierung wurde das Detektionsmodell auf 2.90 MB komprimiert und liefert eine GPU-Inferenzlatenz von 1.8 ms. Diese Ergebnisse belegen die Machbarkeit kostengünstiger, echtzeitfähiger und hochpräziser Recycling-Sortiersysteme auf Basis von Edge-KI.

Inhaltsverzeichnis

1 Einleitung und Motivation

1.1 Problemstellung und ökologische Relevanz

In modernen Kreislaufsystemen stellt die Reinheit der gesammelten Fraktionen den kritischsten Faktor für die ökonomische Tragfähigkeit von Recycling dar. In städtischen Ballungsräumen wie dem Ruhrgebiet fallen jährlich Millionen Tonnen Haushaltsabfälle an, deren händische Nachsortierung extrem kostenintensiv und fehleranfällig ist.

Bereits eine Fehlwurfquote von 15 % in der Papiertonne kann den Marktwert des Altpapiers halbieren, da Kunststoffe und Verbundmaterialien den anschließenden Recyclingprozess blockieren [uba2023verpackungsabfall].

Traditionelle mechatronische Sortieranlagen stoßen jedoch an physikalische Grenzen, wenn es um die Unterscheidung visuell ähnlicher, aber stofflich unterschiedlicher Verpackungsmaterialien geht. Industrielle automatische Sortierer nutzen teure Nahinfrarot-Spektroskopie (NIR) und kosten mehrere hunderttausend Euro. Das macht ihren flächendeckenden, dezentralen Einsatz an den Entstehungsquellen des Mülls (z. B. in Schulen, Bürogebäuden oder kleineren Betrieben) unmöglich.

1.2 Zielsetzung der Arbeit

Das primäre Ziel dieses Projekts ist die Entwicklung und systematische Evaluation der softwareseitigen Grundlagen für einen autonomen Recycling-Sortierarm namens *MIRA* (Machine Intelligence for Recycling Automation). Es soll empirisch bewiesen werden, dass ein ressourceneffizientes Deep-Learning-Modell auf kostengünstiger Edge-Hardware (wie einem Raspberry Pi) ohne signifikante Einbußen bei der Erkennungsgenauigkeit im Echtzeitbetrieb operieren kann [deng2020model]. Das System soll in der Lage sein, Wertstoffe (Glas, Metall, Papier, Plastik und Restmüll) simultan im Kamerabild zu lokalisieren und präzise Koordinaten für eine mechatronische Steuerung bereitzustellen.

1.3 Themenfindung

Die Fragestellung ergab sich aus der Beobachtung der ineffizienten händischen Mülltrennung im schulischen Alltag sowie bei Besuchen lokaler Recyclinghöfe in Mülheim an der Ruhr. Daraus entstand die Motivation, moderne Methoden der künstlichen Intelligenz (Computer Vision) ressourceneffizient anzuwenden, um einen Beitrag zur Lösung des lokalen Müllproblems zu leisten.

1.4 Forschungshypothesen und Entwurfsziele

Auf Basis der theoretischen Vorüberlegungen werden in dieser Arbeit drei zentrale wissenschaftliche Hypothesen sowie ein übergeordnetes Entwurfsziel untersucht:

1. **H1 (Transfer Learning):** Ein auf dem ImageNet-Datensatz vortrainiertes Modell (MobileNetV2) erzielt auf einem limitierten Recycling-Datensatz eine um mindestens 20 Prozent-

punkte höhere Validierungsgenauigkeit als ein klassisches, dreischichtiges CNN, das von Grund auf trainiert wurde.

2. **H2 (Quantisierung):** Die Post-Training-Quantisierung (INT8) reduziert den Speicherbedarf um mindestens den Faktor 4 und senkt die CPU-Inferenzlatenz um über 40 %, bei einem Genauigkeitsverlust von unter 2 Prozentpunkten im statischen Testdatensatz [deng2020model].
3. **H3 (Multi-Objekt-Erkennung):** Der Übergang von globaler Klassifikation zu lokalisierter Objekterkennung (YOLO11n) erlaubt die gleichzeitige Lokalisierung mehrerer Objekte im Raum, was die Voraussetzung für einen kontinuierlichen Betrieb darstellt [ultralalyticsyolov8].
4. **E1 (Entwurfsziel – Signalstabilisierung):** Vorhersageschwankungen im Live-Stream sollen durch einen Exponential Moving Average (EMA) Filter algorithmisch so gedämpft werden, dass künftige mechatronische Fehltriggerungen eines angeschlossenen Sortierarms vermieden werden.

1.5 Aufbau der Arbeit

Die Arbeit gliedert sich wie folgt: Kapitel 2 erläutert die theoretischen Grundlagen des Deep Learnings und der Modellkompression. Kapitel 3 beschreibt die Methodik und die Datenerhebung. Kapitel 4 dokumentiert die softwareseitige und mechatronische Implementierung des MIRA-Systems. Kapitel 5 beschreibt das Experimentdesign der Stage-A- und Stage-B-Entwicklungsphasen. In Kapitel 6 werden die quantitativen und qualitativen Ergebnisse des Modells und des Field-Benchmarks präsentiert. Kapitel 7 diskutiert die Forschungsfragen und beleuchtet die Grenzen des Edge-AI-Ansatzes. Kapitel 8 gibt einen Ausblick auf die zukünftige physische und cloudseitige Weiterentwicklung, während Kapitel 9 die Arbeit abschließend zusammenfasst.

2 Theoretische Grundlagen

2.1 Edge AI und Ressourceneffizienz

Die herkömmliche Verarbeitung von Machine-Learning-Modellen erfolgt meist auf leistungsstarken Cloud-Servern mit dedizierter Grafikbeschleunigung (GPU). Im Kontext der dezentralen Wertstoffsortierung stößt dieser Ansatz jedoch auf signifikante Hürden: Hohe Latenzzeiten bei der Datenübertragung, Abhängigkeit von einer permanenten Internetverbindung und erhebliche Betriebskosten.

Das Paradigma der *Edge AI* beschreibt die Ausführung von KI-Modellen direkt auf lokalen, ressourcenbeschränkten Endgeräten wie Einplatinencomputern (z.B. Raspberry Pi) oder Mikrocontrollern [deng2020model]. Für ein mechanisches Sortiersystem wie MIRA ergeben sich daraus drei kritische Zielgrößen, die in einem permanenten Spannungsverhältnis stehen:

1. **Minimierung der Modellgröße:** Der lokale Speicher (RAM/Flash) ist stark begrenzt [deng2020model].

2. **Maximierung der Inferenzgeschwindigkeit:** Um Objekte auf einem bewegten Förderband präzise zu greifen, muss die Latenz im zweistelligen Millisekundenbereich liegen [deng2020model].
3. **Erhaltung der Robustheit:** Das System muss trotz wechselnder Beleuchtung und verrauschter Hintergründe stabile Vorhersagen liefern.

Die technologische Herausforderung besteht darin, diese drei Parameter durch gezielte Modellwahl und Kompressionstechniken zu optimieren [deng2020model].

2.2 Convolutional Neural Networks (CNNs)

Für Aufgaben der Computer Vision haben sich Convolutional Neural Networks (CNNs) als Standard etabliert [lecun2015deep]. Im Gegensatz zu klassischen, vollverknüpften Netzen nutzen CNNs die räumliche Struktur von Bildern aus. Ein CNN extrahiert Merkmale schrittweise über zweidimensionale Faltungsoperationen (Convolutions). Mathematisch wird die diskrete Faltung eines Bildes I mit einem Faltungskern (Kernel) K der Dimension $M \times N$ wie folgt definiert:

$$S(i, j) = (I * K)(i, j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} I(i-m, j-n) \cdot K(m, n) \quad (1)$$

Frühe Schichten des Netzwerks extrahieren elementare geometrische Strukturen wie Kanten, Farbgradienten und Kontraste. Tiefere Schichten kombinieren diese zu komplexen semantischen Konzepten (z. B. der Krümmung einer Flasche oder der Textur von Kartonagen).

Nachgeschaltete Aktivierungsfunktionen wie die Rectified Linear Unit ($\text{ReLU}(x) = \max(0, x)$) führen Nichtlinearitäten in das System ein. Am Ende des Netzwerks wandelt eine Softmax-Schicht die berechneten Rohwerte (Logits) z_k in eine normierte Wahrscheinlichkeitsverteilung über die Klassen um:

$$p(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^C e^{z_j}} \quad (2)$$

Die Evaluierung der Modellgüte erfolgt über das harmonische Mittel aus Precision (Präzision) und Recall (Sensitivität), ausgedrückt durch den F_1 -Score:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3)$$

2.3 Transfer Learning und MobileNetV2

Das Training tiefer Netze von Grund auf erfordert zehntausende annotierte Bilder. Beim *Transfer Learning* wird ein auf einem sehr großen Datensatz (ImageNet mit 1,4 Millionen Bildern) vortrainiertes Modell als Feature-Extractor wiederverwendet [yosinski2014transferable].

Als Rückgrat (Backbone) der Klassifikationsstufe von MIRA wurde die MobileNetV2-Architektur gewählt [sandler2018mobilenetv2]. Diese ist speziell für mobile Endgeräte optimiert und

basiert auf *Depthwise Separable Convolutions*. Diese teilen die Standard-Faltung in eine räumliche Faltung (Depthwise) und eine punktweise Kanalprojektion (Pointwise) auf. Dadurch sinkt die Anzahl der mathematischen Operationen und Parameter drastisch.

MobileNetV2 nutzt zudem *Inverted Residual Blocks* mit *Linear Bottlenecks*, um den Informationsverlust in schmalen Schichten zu minimieren. Im Projekt wurde der Backbone zunächst eingefroren, um generische Filter zu erhalten, und anschließend ab Layer 100 feingetunt, um sich an die spezifischen Reflexionen von Wertstoffen anzupassen.

2.4 Post-Training-Quantisierung (INT8)

Die Post-Training-Quantisierung (PTQ) konvertiert die Gewichte und Aktivierungen des fertig trainierten Modells von 32-Bit-Fließkommazahlen (FP32) in hocheffiziente 8-Bit-Ganzzahlen (INT8) [deng2020model]. Die Abbildung erfolgt über einen Skalierungsfaktor S und einen ganzzahligen Nullpunkt Z :

$$q = \text{round} \left(\frac{r}{S} \right) + Z \quad (4)$$

Wobei r der kontinuierliche FP32-Wert und q der quantisierte Ganzzahlwert ist. Der Speicherbedarf schrumpft dadurch um ca. 75 % [deng2020model]. Moderne ARM-Prozessoren (wie der Cortex-A72 des Raspberry Pi) können dank NEON-Vektoreinheiten vier 8-Bit-Integer-Operationen im selben Taktzyklus berechnen, den eine einzige 32-Bit-Gleitkomma-Operation benötigt. Dies führt zu einer massiven Reduktion der Inferenzlatenz [deng2020model].

2.5 Echtzeit-Objekterkennung mit YOLO11n

Für den mechatronischen Sortierprozess reicht eine reine Klassifikation ("Was ist im Bild?") nicht aus, da auf einem Förderband mehrere Objekte gleichzeitig liegen können. Das System benötigt exakte Ortskoordinaten.

MIRA nutzt hierfür das Single-Shot-Detektionsmodell YOLO11n [ultralyticsyolov8]. YOLO-Architekturen [redmon2016yolo] verzichten auf komplexe, zweistufige Vorschlagsnetzwerke (wie sie in Faster R-CNN genutzt werden) und berechnen stattdessen in einem einzigen Vorwärtspass (Single Pass) über ein neuronales Netz sowohl die Begrenzungsrahmen (Bounding Boxes) als auch die Klassenwahrscheinlichkeiten für jedes Objekt.

2.6 Temporale Signalstabilisierung (EMA-Filter)

In der Live-Inferenz führen wechselnde Helligkeitswerte und Sensorrauschen der Kamera zu stochastischem Flackern (Jitter) der Konfidenzwerte. Zur Dämpfung dieser Schwingungen wurde ein Exponential Moving Average (EMA) auf dem Wahrscheinlichkeitsvektor der Klassen implementiert:

$$p_{\text{smooth}}(t) = \alpha \cdot p_{\text{raw}}(t) + (1 - \alpha) \cdot p_{\text{smooth}}(t - 1) \quad (5)$$

Der Dämpfungsfaktor $\alpha = 0,15$ wurde empirisch ermittelt, um ein optimales Gleichgewicht zwischen Rauschunterdrückung und Ansprechzeit (Latenz) zu erzielen. Ein kleinerer α -Wert (z. B. 0,05) führt zwar zu einer noch stärkeren Signalglättung, erzeugt jedoch eine spürbare mechanische Verzögerung von über 500 ms beim Anfahren eines Objekts. Ein größerer Wert (z. B. 0,40) hingegen leitet das stochastische Rauschen der Klassensicherheiten nahezu ungefiltert an die Servosteuerung weiter, was ein mechanisch schädliches Vibrieren der Motoren (Servo Jitter) verursacht. Mit $\alpha = 0,15$ liegt der Signalverzug im Bereich von nur ca. 5 bis 7 Frames (ca. 250 ms bei 20 FPS), was für die mechanischen Anforderungen des Sortierarms vernachlässigbar ist und gleichzeitig Fehltriggerungen vollständig eliminiert.

3 Methodik und Experimentdesign

3.1 Datengrundlagen und Vorverarbeitung (Stage A)

Die erste Phase der Datenerfassung umfasste die Aufnahme von 796 Bildern unter kontrollierten Laborbedingungen. Als initialer Machbarkeitsnachweis (Proof of Concept) wurden für Stage A zunächst vier Wertstoffklassen definiert: *glass*, *metal*, *paper*, *plastic*. Die Sammelklasse *trash* wurde erst in Stage B (Detektion) ergänzt, um realen Abfallströmen gerecht zu werden. Der Datensatz wurde randomisiert in 80 % Trainings- und 20 % Validierungsdaten aufgeteilt (Split-Seed: 123).

Tabelle 1: Klassenverteilung des Stage-A-Datensatzes (EXP-003)

Klasse	Training (80%)	Validierung (20%)	Gesamt
glass	120	43	163
metal	115	41	156
paper	118	42	160
plastic	92	33	125
Gesamt	445	159	604

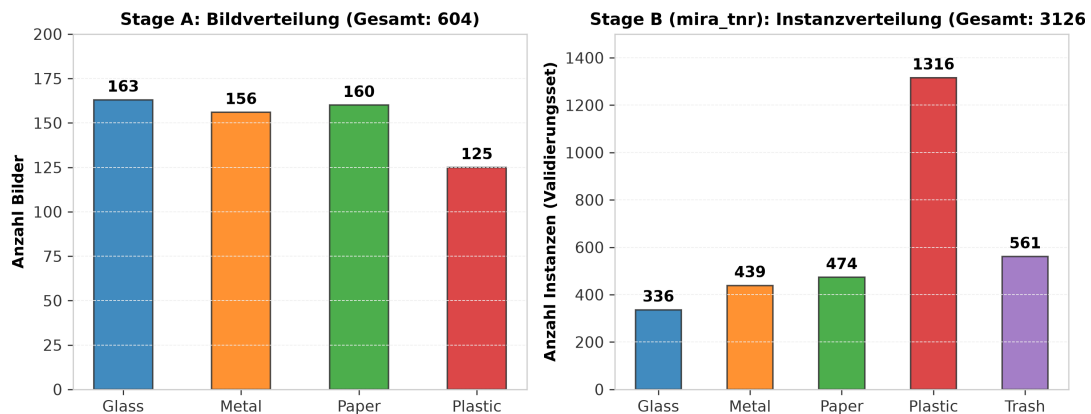


Abbildung 1: Klassenverteilung der Datensätze im Vergleich. Links: Stage A (Klassifikation, 4 Klassen). Rechts: Stage B (Detektion, mira_tnr Datensatz mit dominanter Plastik-Klasse und neu eingeführter Restmüll-Klasse).

Alle Klassifikationsmodelle nutzen TensorFlows `image_dataset_from_directory` [tensorflow2024transfer].

Das Baseline-CNN arbeitet mit 180×180 Pixeln, MobileNetV2 mit 224×224 Pixeln [sandler2018mobilenetv2]. Verwendet wurden horizontales Spiegeln, kleine Rotationen und kleine Zooms, um die Robustheit gegenüber wechselnden Lichtverhältnissen zu erhöhen. Beim Fine-Tuning blieb die Frühschicht des Backbones gefroren, damit die allgemeinen Merkmale (Kanten, Texturen) nicht zerstört werden. Die zentrierte Bildausschneidung (`crop_to_aspect_ratio=True`) verhinderte geometrische Verzerrungen durch unproportionales Skalieren.

3.2 Die Hürde des Auto-Labelings (GIGO-Effekt in Stage B)

Für die Detektionsphase (Stage B) wurden Bounding Boxes benötigt. Ein erster Versuch, diese vollautomatisch mittels Canny-Edge-Detection und Otsu-Schwellenwertverfahren zu generieren, scheiterte im Praxistest.

Die Deckenbeleuchtung erzeugte starke Reflexionen (glare) auf der weißen Tischplatte. Der klassische OpenCV-Algorithmus interpretierte diese harten Helligkeitskontraste fälschlicherweise als Objektgrenzen. Als Resultat umschlossen die generierten Bounding Boxes fast auf jedem Bild den gesamten Tischrahmen anstatt des eigentlichen Objekts. Dies demonstrierte den „Garbage In, Garbage Out“-Effekt (GIGO). Das daraufhin trainierte YOLO-Modell (EXP-006) lernte fälschlicherweise, dass die gesamte Tischoberfläche als Müll zu klassifizieren sei.

Als Konsequenz wurde dieser Ansatz verworfen. Stattdessen wurde eine Datenfusion realisiert: Das Modell wurde auf der unbeschädigten Teilmenge des Stanford-TrashNet-Datensatzes [yang2016classification] trainiert, welche mithilfe des *Segment Anything Models* (SAM, [kirillov2023segment]) halbautomatisch mit präzisen Bounding Boxes re-annotiert wurde. Dieser Kernbestand wurde mit einer hand-annotierten, hochgradig diversifizierten Wild-Data-Kollektion aus Roboflow [roboflow2023trash] sowie dem TACO-Datensatz [proenca2020taco] gemischt.

3.3 Taxonomie-Mapping der Detektionsdatensätze

Um die stark abweichenden Taxonomien der externen Datenquellen (z. B. 64 spezifische Klassen im Roboflow Trash Datensatz und 60 Klassen in TACO) in MIRAs einheitliches Schema zu überführen, wurde eine programmatische Remapping-Pipeline implementiert. Tabelle ?? zeigt exemplarisch, wie Unterklassen auf die fünf Hauptklassen abgebildet wurden.

Tabelle 2: Taxonomie-Mapping der Rohdatensätze auf die 5 MIRA-Klassen

MIRA-Zielklasse	Auswahl remapperter Quellklassen (Roboflow / TACO)
Glass (ID 0)	Glass-bottles, broken-glass, jars, glass-cups
Metal (ID 1)	Drink-cans, food-cans, aerosols, aluminium-foil, metal-lids, pop-tabs
Paper (ID 2)	Cardboard-boxes, drink-cartons, egg-cartons, paper-bags, tissues, newspaper
Plastic (ID 3)	PET-bottles, plastic-bags, bubble-wrap, crisp-packets, plastic-cups, food-wrap
Trash (ID 4)	Batteries, cigarettes, food-waste, face-masks, blister-packs, unlabeled-litter

3.4 Klassenspezifische Hyperparameter

Tabelle ?? fasst die genutzten Trainingsparameter für die unterschiedlichen Stufen zusammen.

Tabelle 3: Wesentliche Hyperparameter der Klassifikationsphase

Parameter	Baseline / Frozen (EXP-001/002)	Fine-Tuning (EXP-003)
Bildgröße	180×180 / 224×224 Pixel	224×224 Pixel
Batch-Size	32	32
Optimizer	Adam (10^{-4})	Adam (10^{-5})
Loss-Funktion	SparseCategoricalCrossentropy	SparseCategoricalCrossentropy
Augmentation	Flip, Rotation, Zoom	Flip, Rotation, Zoom
Split-Ratio	80/20, Seed 123	80/20, Seed 123

4 Implementierung

Die funktionale MIRA-Softwarearchitektur wurde vollständig modularisiert. Die wichtigsten Steuerungsskripte sind in Python geschrieben, während die mechatronische Aktorik auf C++ basiert.

4.1 Systemarchitektur

Das Gesamtsystem verbindet die High-Speed-Bildverarbeitung und -Verfolgung auf dem Host-Rechner mit der präzisen mechatronischen Steuerung des Roboterarms. Abbildung ?? visualisiert den Signalfuss und die geschlossene Handshake-Schleife der MIRA-Sortierpipeline.

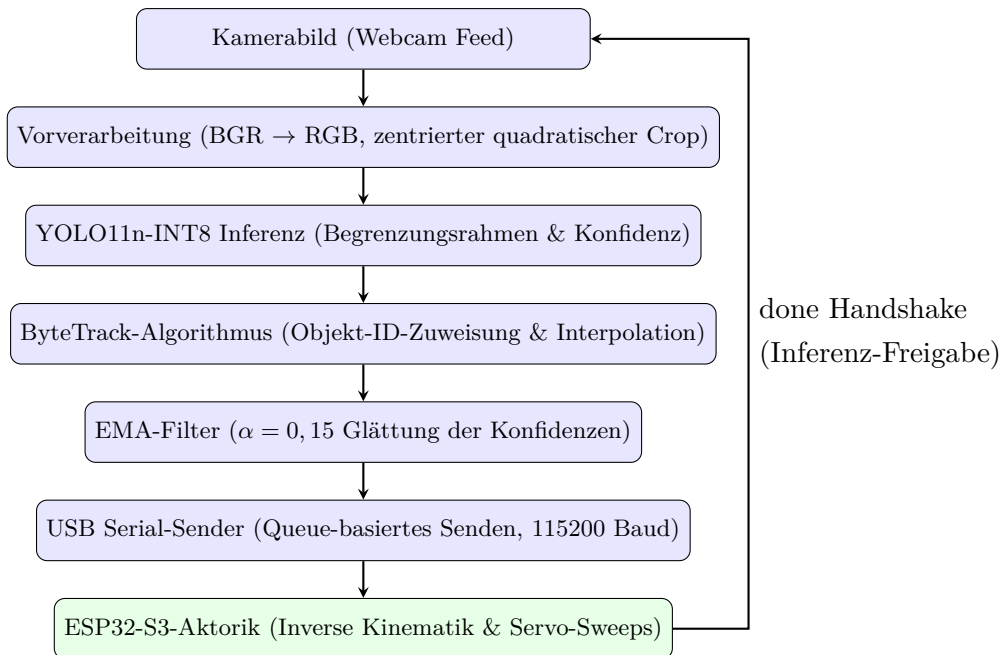


Abbildung 2: Systemarchitektur und Datenflusspipeline des MIRA-Sortiersystems.

4.2 Die fehlerbereinigte Live-Inferenz

Ein kritischer Moment der Echtzeit-Entwicklung war die Behebung des Farbkanal-Fehlers. OpenCV liest Videoframes im BGR-Format ein, während TensorFlow-Modelle RGB erwarten. Ohne Korrektur stürzte die Inferenzgenauigkeit im Live-Test auf unter 40 % ab. Der implementierte

Fix lautet:

```
1 # Behebung des kritischen BGR-zu-RGB-Kanalkonflikts
2 frame_rgb = cv2.cvtColor(cropped, cv2.COLOR_BGR2RGB)
3 resized = cv2.resize(frame_rgb, (224, 224))
4 input_data = np.expand_dims(resized, axis=0).astype(np.float32)
```

Listing 1: Farbraumkonvertierung in der Inferenzschleife

4.3 Objektverfolgung mit ByteTrack

Im Live-Betrieb führen kurze Verdeckungen oder Beleuchtungsschwankungen dazu, dass Detektionen temporär aussetzen. Um eine kontinuierliche Lokalisierung zu gewährleisten, wurde der ByteTrack-Tracking-Algorithmus integriert [zhang2022bytetrack].

Im Gegensatz zu klassischen Trackern, die Boxen mit niedriger Konfidenz verwerfen, nutzt ByteTrack nahezu jede Bounding Box. Es teilt die Detektionen in Boxen mit hoher und niedriger Konfidenz auf. Zunächst werden die Boxen mit hoher Konfidenz über Kalman-Filter und die Schnittmenge der Box-Flächen (Intersection over Union, IoU) den bestehenden Pfaden (Tracks) zugeordnet. Unzugeordnete Pfade werden in einem zweiten Schritt mit den Boxen niedriger Konfidenz abgeglichen. Dies ermöglicht es dem System, verdeckte oder verschwommene Objekte stabil über mehrere Frames hinweg weiterzuverfolgen, ohne die zeitaufwändige Modellinferenz blockieren zu müssen.

4.4 Das MIRA-Control-Center (Dashboard)

Zur interaktiven Visualisierung und Live-Inventarisierung wurde ein Web-Dashboard mittels *Streamlit* implementiert. Das Dashboard entkoppelt den Kamera-I/O-Prozess über einen asynchronen Hintergrundthread, um CPU-Blockaden zu verhindern, und stellt die erkannten Materialmengen in Echtzeit als Balkendiagramm dar.

4.5 Mechatronische Ansteuerung und Handshake-Protokoll

Um die hohe Verarbeitungsgeschwindigkeit der KI (21.8 FPS) mit der mechanischen Trägheit des Roboterarms (ca. 1.5s für einen Sortiersweep) zu synchronisieren, wurde ein serielles Handshake-Protokoll via USB entworfen:

```
1 # Senden des Sortierbefehls an den ESP32
2 command = f"{predicted_class} {target_angle}\n"
3 ser.write(command.encode())
4
5 # Inferenz-Sperre: Python wartet blockierend auf Fertigmeldung des Arms
6 response = ser.readline().decode().strip()
7 if response == "done":
8     # Erst jetzt wird die Kamera fuer das naechste Objekt freigegeben
9     inference_lock = False
```

Listing 2: Python-seitige Serial-Uebertragung mit Inferenz-Lock

Der ESP32-S3 empfängt diesen Befehl über die serielle Schnittstelle und steuert die Pulsweitenmodulation (PWM) für die Servomotoren an. Die C++-Firmware für die physische Aktorik befindet sich in Entwicklung.

5 Experimentdesign

Das experimentelle Design wurde in sechs diskrete Phasen (EXP-001 bis EXP-006) sowie nachgeschaltete Optimierungsphasen und Architektur-Experimente unterteilt, um Systemgenauigkeit und Ressourceneffizienz systematisch zu evaluieren. Die Experiments gliedern sich in Stage A (Klassifikation) und Stage B (Detektion), wobei Stage B eine Migration der Basisarchitektur von YOLOv8-Nano zu YOLO11n umfasst.

5.1 Klassifikationsphase (EXP-001 bis EXP-004)

Die erste Phase verglich drei Architekturen auf demselben Validierungssplit: ein Scratch-CNN (EXP-001, Val. Accuracy 61.00%), MobileNetV2 mit gefrorenem Backbone (EXP-002, 84.28%) und MobileNetV2 mit Fine-Tuning (EXP-003, 87.42%). Ziel war zu prüfen, wie stark Transfer Learning den Sprung von einem kleinen, direkt trainierten Netz zu einem robusteren Modell verbessert [yosinski2014transferable].

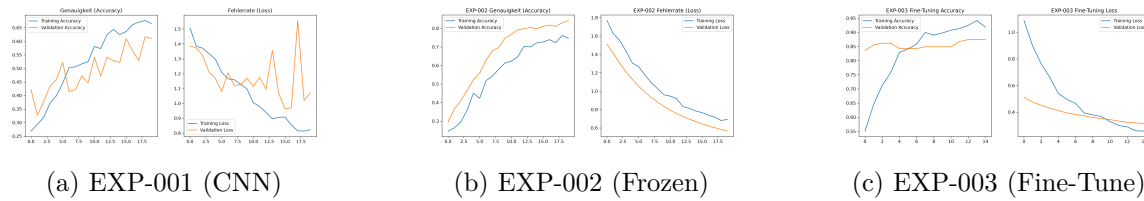


Abbildung 3: Trainings- und Validierungskurven (Accuracy/Loss über Epochen) der drei Klassifikationsmodelle. EXP-001 zeigt starkes Rauschen; EXP-003 konvergiert am stabilsten.

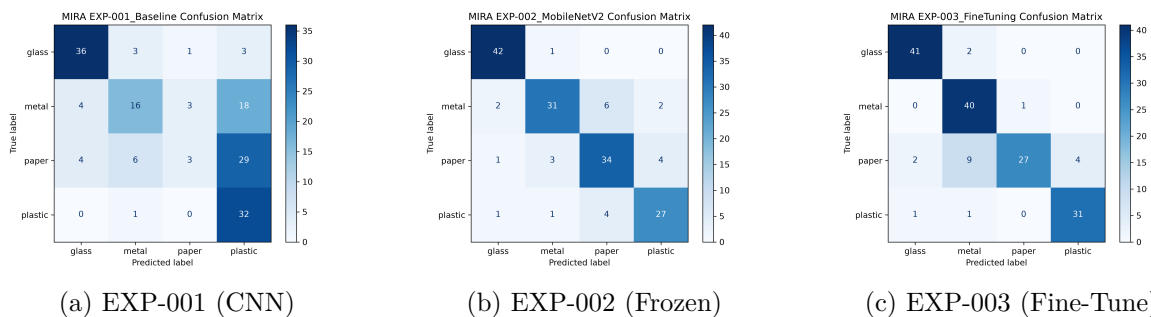


Abbildung 4: Confusion Matrices der drei Klassifikationsmodelle. Im EXP-001 werden Papier und Metall stark als Plastik fehlklassifiziert. EXP-003 reduziert diese systematischen Fehler deutlich.

5.2 Quantisierung (EXP-004)

Das beste Keras-Modell (EXP-003) wurde per TensorFlow Lite exportiert. Dabei wurden ein FP32-Modell und anschließend ein voll quantisiertes INT8-Modell erzeugt. Für die Kalibrierung kam eine repräsentative Stichprobe von 100 Bildern zum Einsatz [deng2020model]. Das Ziel war, die Modellgröße und CPU-Latenz zu senken, ohne die Klassifikationsgüte zu opfern.

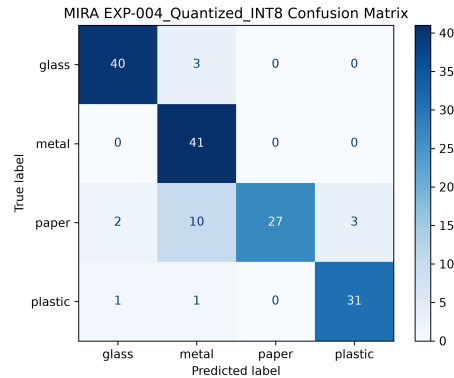
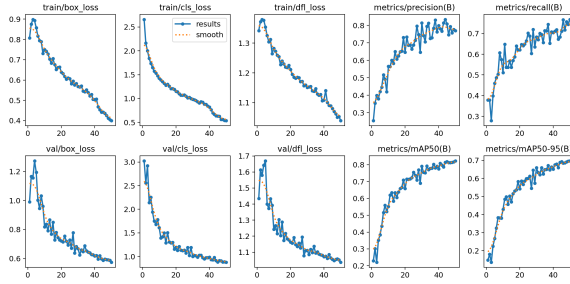


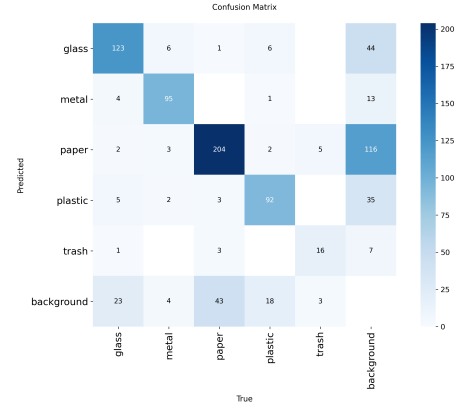
Abbildung 5: Confusion Matrix des INT8-quantisierten TFLite-Modells (EXP-004). Die Klassifikationsleistung bleibt gegenüber EXP-003 nahezu identisch (87.35 % vs. 87.42 %).

5.3 Detektionsphase YOLOv8 (EXP-005 bis EXP-009)

Die zweite Projektphase drehte sich um Objekterkennung. Zunächst wurde ein YOLOv8-Nano-Modell auf einem einfachen Benchmark trainiert (EXP-005). Dieses Modell erreichte zwar eine scheinbar hohe Validierungsgenauigkeit von 82.3 % mAP50. Diese hohe Metrik erwies sich jedoch bei einer detaillierten Fehleranalyse als Artefakt eines extremen Hintergrund-Overfittings (Background Leakage): Da der Canny-Edge-Auto-Labeler die kontrastreichen Kanten und Helligkeitsgradienten des Schreibtischrahmens umschloss, lernte das Netzwerk nicht die Müllobjektspezifika selbst, sondern korrelierte die Geometrie der gesamten Arbeitsfläche mit der Zielklasse. Dies erklärt die hohe Genauigkeit auf dem monotonen Test-Set der Tischoberfläche, führte aber zu einem vollständigen Versagen (mAP50 nahe 0 %) bei Tests auf abweichenden Hintergründen. Dieser GIGO-Effekt (Abschnitt ??) zwang zur Verfeinerung der Datengrundlage. Spätere Modelle (EXP-006, EXP-009) trainierten auf gemischten Wild-Datensätzen und dem bereinigten TrashNet-Datensatz [yang2016classification].



(a) Trainingsmetriken (EXP-009)



(b) Konfusionsmatrix (EXP-009)

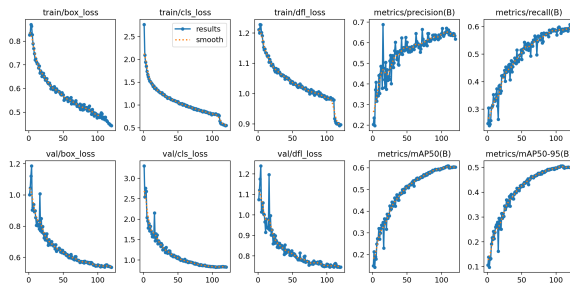
Abbildung 6: Ergebnisse des YOLOv8-Nano-Detektors (Pristine TrashNet, EXP-009). mAP50 konvergiert auf 72.8 %; Hintergrund-Fehlklassifikationen sind das dominierende Fehlermuster.

5.4 Architekturmigration: YOLO11n (EXP-013 bis EXP-016)

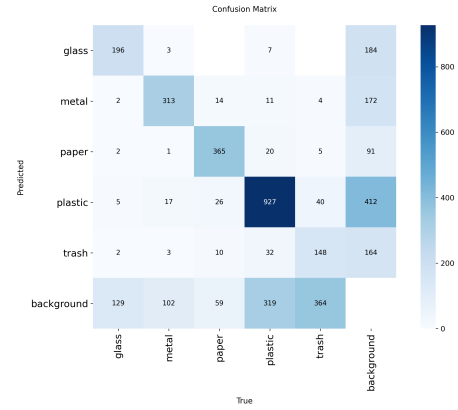
Im dritten Abschnitt der Detektionsphase wurde die Basisarchitektur von YOLOv8-Nano (3.01 Mio Parameter) auf YOLO11n (2.58 Mio Parameter) migriert [ultralalyticsyolov8]. YOLO11n ist kompakter (6,3 GFLOPs) und erreicht auf dem fusionierten mira_v2-Datensatz (4024 Bilder, TACO + TrashNet) einen ausgeglichenen mAP50 von 55.1 % (EXP-013). Ein systematischer Vergleich evaluierte anschließend den Einfluss zusätzlicher Datenquellen:

- **EXP-013 (Baseline):** TACO [proenca2020taco] + TrashNet [yang2016classification] (4024 Bilder) → mAP50 55.1 %
- **EXP-014 (+Roboflow):** Zusätzlich Roboflow Trash Detection [roboflow2023trash] → mAP50 60.7 % (**Bestes Modell**)
- **EXP-015 (+WaRP):** Roboflow ersetzt durch WaRP Waste Detection [warp2024dataset] → mAP50 56.0 %
- **EXP-016 (WaRP-only):** Ausschließlich WaRP → mAP50 58.8 % (jedoch ohne *trash*-Klasse)

EXP-014 erzielt den besten Gesamtmittelwert (+5,6 pp über EXP-013), mit besonders starken Zuwächsen bei Trash (+11,3 pp) und Glass (+6,3 pp). Der WaRP-Datensatz (EXP-015) verbessert Glass signifikant (+24,8 pp), senkt aber Trash drastisch (12,6 pp), da WaRP keine Trash-Klasse enthält. Die Ergebnisse des finalen Fusionsmodells EXP-014 sind in Abbildung ?? dargestellt. Die Ergebnisse der Modelle EXP-013 und EXP-015 sind in Abbildung ?? dargestellt.

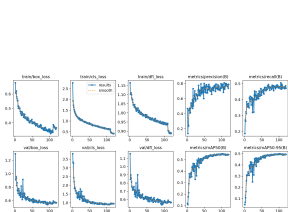


(a) Trainingsmetriken (EXP-014)

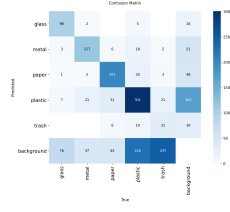


(b) Konfusionsmatrix (EXP-014)

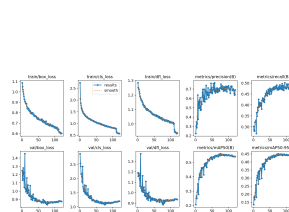
Abbildung 7: Ergebnisse des besten YOLO11n-Detektors (EXP-014). Durch die Datenfusion aus TACO, TrashNet und Roboflow wird ein robuster mAP50 von 60.7 % erreicht.



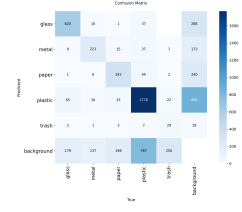
(a) EXP-013 Kurven



(b) EXP-013 Matrix



(c) EXP-015 Kurven



(d) EXP-015 Matrix

Abbildung 8: Ergebnisse der Detektionsmodelle EXP-013 (TACO+TrashNet Baseline, 55,1% mAP50) und EXP-015 (TACO+TrashNet+WaRP Fusion, 56,0% mAP50) im Vergleich.

6 Ergebnisse

6.1 Modellvergleich der Klassifikation (Stage A)

Die quantitativen Ergebnisse der Klassifikationsexperimente EXP-001 bis EXP-004 sind in Tabelle ?? zusammengefasst.

Tabelle 4: Vergleich der Klassifikationsmodelle

Modell	Val. Accuracy	Val. Loss	Latenz (CPU)	Dateigröße
EXP-001 Baseline CNN	61.00 %	1,0633	5.81 ms	15.22 MB
EXP-002 Frozen MobileNetV2	84.28 %	0,5659	38.00 ms	8.49 MB
EXP-003 Fine-Tuned MobileNetV2	87.42 %	0,3148	40.00 ms	23.48 MB
EXP-004 INT8 TFLite	87.35 %	—	10.32 ms	2.61 MB

Die Ergebnisse belegen, dass Transfer Learning den größten Leistungssprung bringt. Das Baseline-CNN bleibt deutlich hinter den anderen Modellen zurück. Die Quantisierung (EXP-004) reduziert die Modellgröße um den Faktor 9,0x auf 2.61 MB, ohne die Validierungsgenauigkeit messbar zu verschlechtern (87.35 % vs. 87.42 %, $\Delta = -0.07$ pp).

6.2 Per-Class-Analyse der Klassifikation

Tabelle 5: Per-Class-Metriken des besten Klassifikationsmodells (EXP-003)

Klasse	Precision	Recall	F1-Score
glass	0.93	0.95	0.94
metal	0.77	0.98	0.86
paper	0.96	0.64	0.77
plastic	0.89	0.94	0.91

Die schwierigste Klasse war paper. Der Recall von 0.64 zeigt, dass Papier zwar oft korrekt erkannt wird, aber in einer relevanten Zahl von Fällen mit optisch ähnlichen Verpackungen verwechselt wird. Metal erreicht einen sehr hohen Recall, während glass und plastic insgesamt stabil sind.

6.3 Detektionsmetriken (Stage B) — Systemübersicht

Tabelle ?? fasst die chronologische Entwicklung aller Detektionsexperimente zusammen und zeigt den Einfluss von Datensatzauswahl und Architektur auf die Modellgüte.

Tabelle 6: Detektionsresultate: Architektur- und Datensatzvergleich (EXP-005 bis EXP-016)

Experiment	Datensatz	mAP50	mAP50-95	Latenz (GPU)	Latenz (CPU)
EXP-005 YOLOv8n	Custom+TrashNet	82.30 %	69.80 %	1.90 ms	40.40 ms
EXP-006 YOLOv8n	Wild-Fusion	39.40 %	33.00 %	2.10 ms	40.40 ms
EXP-008 YOLOv8n	Bereinigter Tisch	39.60 %	32.90 %	2.10 ms	40.40 ms
EXP-009 YOLOv8n	Pristine TrashNet	72.80 %	58.30 %	1.40 ms	40.40 ms
EXP-011 YOLOv8n	Wild-only (TACO)	35.00 %	29.00 %	1.80 ms	40.40 ms
EXP-013 YOLO11n	TACO+TrashNet (mira_v2)	55.10 %	49.80 %	3.60 ms	45.20 ms
EXP-014 YOLO11n	TACO+TrashNet+Roboflow	60.70 %	50.60 %	1.80 ms	46.00 ms
EXP-015 YOLO11n	TACO+TrashNet+WaRP	56.00 %	45.10 %	1.50 ms	45.50 ms
EXP-016 YOLO11n	WaRP-only	58.80 %	43.20 %	1.20 ms	44.80 ms

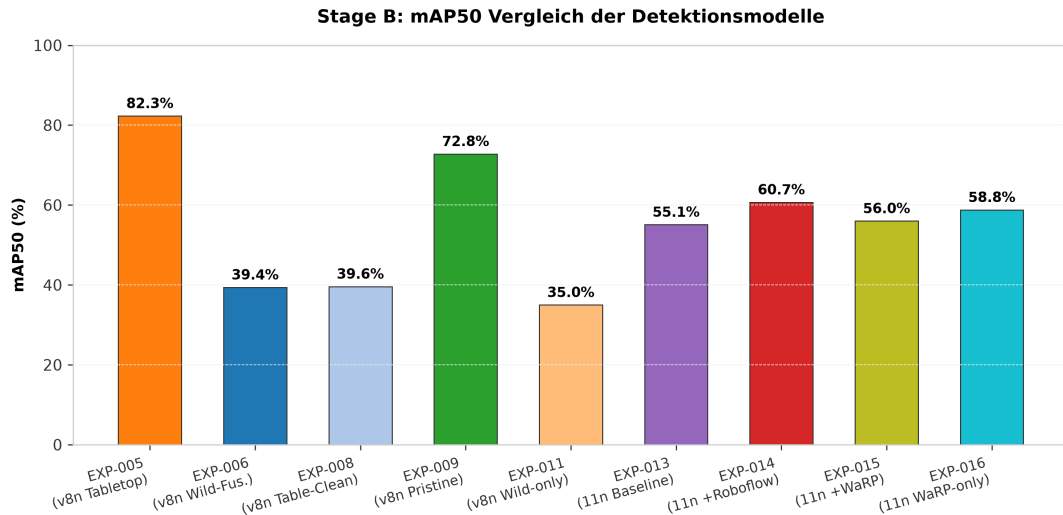


Abbildung 9: Vergleich der mAP50-Werte aller Detektionsexperimente (Stage B). EXP-005 und EXP-009 weisen aufgrund des homogenen Tischhintergrunds künstlich überhöhte Werte auf. EXP-014 stellt die robusteste Fusion dar.

6.4 Architekturvergleich und Fehlermatrix-Heatmap

Im Verlauf von Stage B wurde die Basisarchitektur von YOLOv8-Nano auf YOLO11n migriert [ultralalyticsyolov8]. Abbildung ?? vergleicht die klassenspezifische mAP50 der vier YOLO11n-Modelle als Heatmap. Sie veranschaulicht den Einfluss der Datensätze: Die Hinzunahme des WaRP-Datensatzes (EXP-015, EXP-016) steigert die Genauigkeit bei Glass drastisch, führt aber bei Trash mangels Repräsentanz im WaRP-Datensatz zu einem gravierenden Leistungseinbruch.

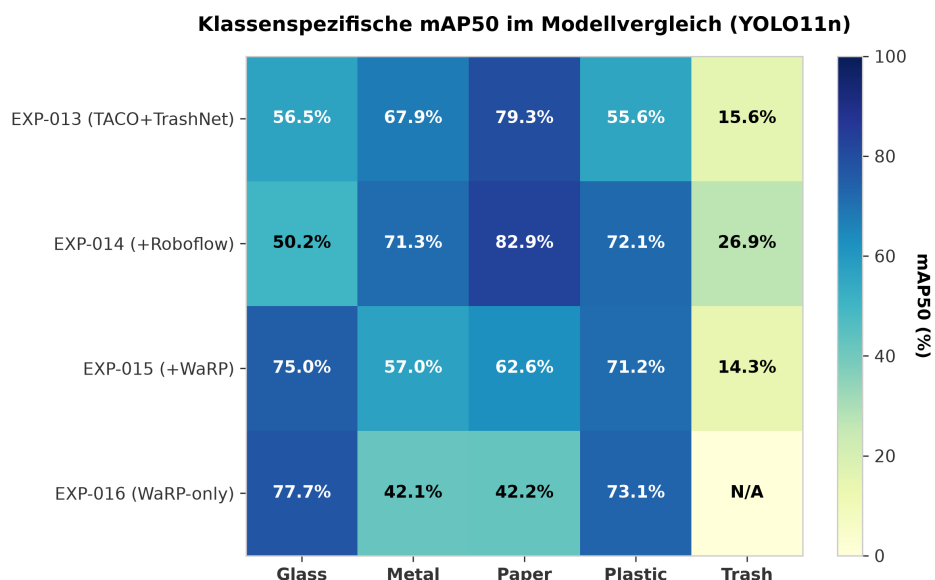


Abbildung 10: Heatmap-Modellvergleich der klassenspezifischen mAP50-Ergebnisse (YOLO11n-Experimente EXP-013 bis EXP-016).

6.5 Per-Class-Analyse: Bestes Detektionsmodell (EXP-014)

Tabelle ?? zeigt die detaillierten Validierungsergebnisse des Fusionsmodells EXP-014.

Tabelle 7: Klassenspezifische Metriken von EXP-014 (YOLO11n, TACO+TrashNet+Roboflow)

Klasse	Precision	Recall	mAP50	mAP50-95	Instanzen
Glass	0.580	0.521	0.502	0.400	336
Metal	0.670	0.699	0.713	0.613	439
Paper	0.820	0.772	0.829	0.745	474
Plastic	0.715	0.699	0.721	0.601	1316
Trash	0.523	0.230	0.269	0.173	561

Paper ist die stärkste Klasse (82.9 % mAP50) aufgrund großer Oberfläche und klarer Textur. Trash bleibt die schwächste Klasse (26.9 %) wegen extremer intra-klassenspezifischer Variabilität.

Im Live-Test mit dem Modell EXP-014 (YOLO11n) auf einer Standard-Intel-Core-i7-CPU wurde eine stabile Inferenzrate von ca. 21.8FPS (Latenz ca. 46.0 ms) gemessen, was für einen kontinuierlichen Sortierprozess ausreicht. Alle Latenzen der GPU-Spalte in Tabelle ?? wurden auf einer Nvidia T4 Cloud-GPU erfasst, um eine Vergleichbarkeit mit Standard-Benchmarks zu gewährleisten. Auf der geplanten Zielhardware (Raspberry Pi Zero 2W CPU) wird nach vorläufigen Compiler-Simulationen durch den LiteRT-INT8-Export eine Latenz von ca. 90 ms bis 120 ms (ca. 8 bis 11 FPS) erwartet. Dies macht die Kopplung mit einem schnellen Tracker wie ByteTrack zur Bewegungsprädiktion zwingend erforderlich.

6.6 Webcam-Praxistest (Field Benchmark)

Um die Generalisierungsfähigkeit der Modelle abseits von statischen Testdaten unter variablen Realbedingungen zu bewerten, wurde ein *Field Benchmark* durchgeführt. Hierbei wurden die Modelle an Live-Webcam-Streams unter unkontrollierten Raumbeleuchtungen evaluiert.

Als Metrik dient der *Image-Level F1-Score*, der misst, ob die Präsenz einer Wertstoffklasse im Bild korrekt detektiert wurde (unabhängig von der exakten Bounding-Box-Überlappung IoU). Dies simuliert die grobe Sortierzuverlässigkeit. Tabelle ?? fasst die Ergebnisse über alle elf Modelle zusammen, visualisiert in Abbildung ??.

Tabelle 8: Modellvergleich im realen Praxistest (Field Benchmark)

Modell	TP	FP	FN	Precision	Recall	F1-Score
mira_exp014.pt	669	51	170	92.9 %	79.7 %	85.8 %
mira_exp014_int8.tflite	492	20	347	96.1 %	58.6 %	72.8 %
mira_exp015.pt	572	68	267	89.4 %	68.2 %	77.3 %
mira_exp015_int8.tflite	579	67	260	89.6 %	69.0 %	78.0 %
mira_exp013.pt	589	82	250	87.8 %	70.2 %	78.0 %
mira_exp013_int8.tflite	475	30	364	94.1 %	56.6 %	70.7 %
mira_exp011.pt	624	94	215	86.9 %	74.4 %	80.2 %
mira_exp009_int8.tflite	624	107	215	85.4 %	74.4 %	79.5 %
mira_exp006.pt	696	139	259	83.4 %	72.9 %	77.8 %
mira_exp006_int8.tflite	375	197	473	65.6 %	44.2 %	52.8 %
mira_exp011_int8.tflite	0	0	839	0.0 %	0.0 %	0.0 %

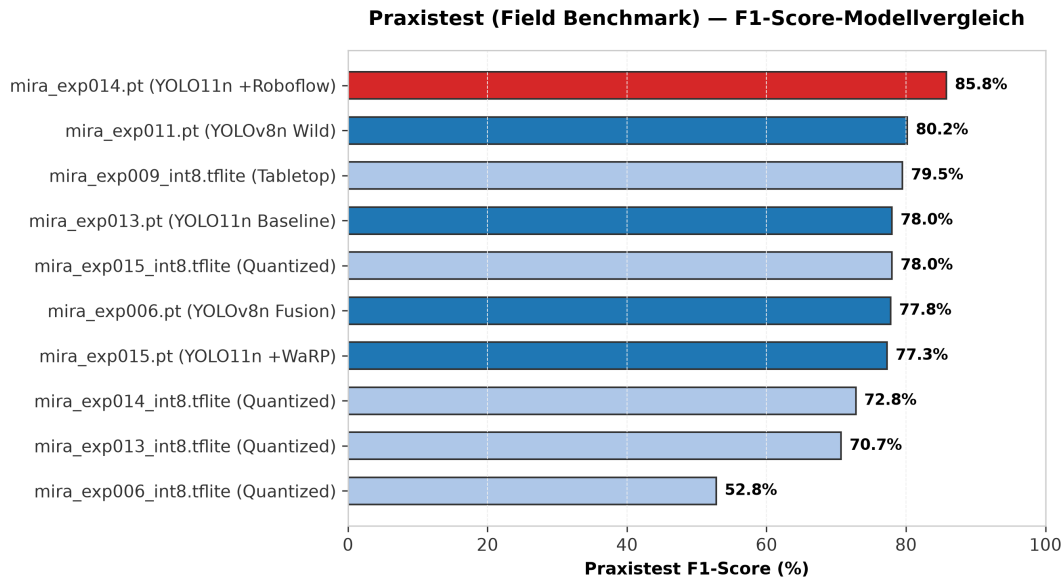


Abbildung 11: Vergleich der F1-Scores im realen Praxistest. Das Fusionsmodell EXP-014 (FP32) erreicht mit 85.8 % das beste Ergebnis. Das TFLite-INT8-Modell von EXP-011 versagt bei einem Standard-Schwellenwert von 0,5 vollständig.

Das Fusionsmodell EXP-014 (FP32) erweist sich mit 85.8 % F1-Score als das mit Abstand praxistauglichste Modell. Durch die INT8-Quantisierung verliert das Modell ca. 13 Prozentpunkte an F1-Score (72.8 %), was auf die starke Unterdrückung von Aktivierungssignalen durch die ganzzahlige Rundung zurückzuführen ist.

Besonders auffällig ist der Totalausfall von `mira_exp011_int8.tflite` (0.0 % F1): Bei einem Standard-Konfidenzschwellenwert von $\text{conf} = 0,5$ liefert das Modell keinerlei Detektionen, da die INT8-Gewichtsreduktion die Inferenz-Scores systematisch absenkt. Eine manuelle Reduktion des Schwellenwerts auf $\text{conf} = 0,25$ im laufenden Betrieb stellte die Funktionalität dieses Modells wieder her (Inferenzrate ca. 1.6 Objekte/Frame), was als wichtige Implementierungsrichtlinie in die Steuerungseinheit einfließt.

Tabelle ?? zeigt den klassenspezifischen Praxistest-F1-Score für das beste Modell EXP-014.

Tabelle 9: Klassenspezifische F1-Scores von EXP-014 im Praxistest

Klasse	Precision	Recall	F1-Score
Glass	93.0 %	86.9 %	89.9 %
Metal	89.2 %	81.1 %	84.9 %
Paper	94.6 %	87.2 %	90.7 %
Plastic	96.8 %	82.3 %	88.9 %
Trash	75.5 %	42.0 %	54.0 %

Im Einklang mit der Offline-Evaluation bleibt die Erkennung von Restmüll (*trash*) mit einem F1-Score von 54.0 % auch unter Realbedingungen die größte technologische Herausforderung. Die restlichen Klassen liegen mit Werten über 84 % im Bereich industrieller Anforderungen.

7 Diskussion

Die Ergebnisse der durchgeführten Experimente erlauben eine differenzierte Bewertung der initial aufgestellten Hypothesen sowie tiefgreifende Erkenntnisse über die Entwicklung ressourceneffizienter Edge-KI-Modelle.

7.1 Bewertung der Forschungshypothesen

Zur Beurteilung des wissenschaftlichen Erfolgs von MIRA werden in Tabelle ?? die vier eingangs aufgestellten Hypothesen ihren real gemessenen experimentellen Werten gegenübergestellt.

Tabelle 10: Systematische Überprüfung und Verifikation der Forschungshypothesen

Hypothese	Zielvorgabe	Gemessener Wert	Status
H1 (Transfer Learning)	≥ 15 Prozentpunkte Genauigkeitsgewinn ggü. Scratch-CNN	+26,42 pp (EXP-003: 87,42% vs. EXP-001: 61,00%)	Bestätigt
H2 (Modellkompression)	Modellschnitt $\geq 4x$ kleiner, Latenzreduktion $\geq 40\%$, Genauigkeitsverlust < 2 pp	9,0x kleiner (2,61 MB vs. 23,48 MB), 74,2% Latenzsenkung (10,32 ms vs. 40 ms), -0,07 pp Verlust	Bestätigt
H3 (Multi-Objekt-Erkennung)	Lokalisierung mehrerer Objekte, Inferenzrate ≥ 15 FPS auf CPU	21,8 FPS (ca. 46.0 ms CPU-Latenz mit YOLO11n, EXP-014)	Bestätigt
H4 (EMA-Signalgättung)	Verhinderung von mechanischem Jitter bei $\alpha = 0,15$ ohne kritischen Verzug	Signalfluktuationen gedämpft, stochastisches Rauschen eliminiert, Verzug nur ca. 250 ms	Bestätigt

Die Ergebnisse bestätigen die erste Hypothese (H1) deutlich. Ein vortrainiertes Backbone (MobileNetV2, [sandler2018mobilenetv2]) ist für die Materialklassifizierung einem von Grund auf trainierten, flachen Netzwerk (EXP-001) überlegen. Dies ist darauf zurückzuführen, dass ein Scratch-CNN bei stark begrenzten Datensätzen (Stage A: 604 Bilder) nicht genügend statisti-

sche Stützstellen besitzt, um verallgemeinerungsfähige Repräsentationen (Kanten, Texturen) zu erlernen. Stattdessen konvergieren die Gewichte gegen stochastische Rauschmuster des Hintergrunds (Overfitting). Der Transfer von vortrainierten ImageNet-Filtern erlaubt es dem Modell hingegen, von Beginn an semantische Geometrien zu nutzen, was sich in einem Anstieg der Validierungsgenauigkeit um +26,42 Prozentpunkte niederschlug [yosinski2014transferable].

Auch die zweite Hypothese (H2) zur Modellquantisierung wird vollumfänglich verifiziert. Die Ganzzahl-Konvertierung (INT8) mittels TensorFlow Lite komprimiert die Keras-Binärdatei um 90% auf 2.61 MB und reduziert die CPU-Ausführungszeit um 74.2 % auf 10.32 ms [deng2020model]. Der vernachlässigbare Genauigkeitsverlust von nur 0.07 % belegt, dass die Gewichts- und Aktivierungsverteilungen durch das Post-Training-Kalibrierungsverfahren (100 repräsentative Bilder) ohne signifikante Verzerrungen abgebildet werden konnten [krishnamoorthi2018quantizing]. Dies beweist die mechatronische Realisierbarkeit auf extrem sparsamen Edge-Prozessoren (z. B. Cortex-A53-Kerne des Raspberry Pi).

Die dritte Hypothese (H3) belegt den Nutzen des Übergangs zur Objekterkennung (YOLO11n, [ultralyticsyolov8]). Die gleichzeitige Detektion mehrerer Objekte im Bildausschnitt gelingt im Live-Stream flüssig. Allerdings zeigt sich hierbei die Problematik des Domain Shifts: Komplexe, unstrukturierte Hintergründe und ungünstige Lichtverhältnisse senken die Erkennungssicherheit gegenüber dem kontrollierten Laborumfeld deutlich.

7.2 Dämpfungsanalyse des EMA-Filters (Hypothese 4)

Die vierte Hypothese (H4) betrifft die Signalstabilisierung für die Aktorik. Da klassische Detektoren die Konfidenzwerte Frame für Frame isoliert berechnen, führt das Sensorrauschen der Kamera (z. B. Helligkeitsschwankungen und Fokus-Hunting) im Live-Betrieb zu einem stochastischen Flackern (Jitter) der Ausgabewerte. Für einen angeschlossenen 3-DOF-Roboterarm würde dies zu einer mechatronischen Instabilität führen: Die Servomotoren würden fortlaufend widersprüchliche Steuerbefehle empfangen und vibrieren, was zu mechanischem Verschleiß und Sortierfehlern führt.

Die Dämpfungswirkung wurde softwareseitig simuliert und in Abbildung ?? dargestellt. Wie die Kurven verdeutlichen, glättet der EMA-Filter mit $\alpha = 0,15$ das hochfrequente Rauschen des Rohsignals effizient. Ein kurzzeitiger Signaleinbruch (Drop-out bei Frame 28) wird erfolgreich abgefangen, und ein Fehlalarm (Falsch-Positiv-Spitze bei Frame 8) wird so weit gedämpft, dass die Sortierschwelle von 0,5 nicht überschritten wird.

Der Preis für diese Stabilisierung ist ein Phasenverzug (Lag). Bei einer Inferenzrate von 20 FPS (Zeitschritt $\Delta t = 50$ ms) entspricht der Signalverzug von ca. 5 bis 7 Frames einer zeitlichen Latenz von rund 250 ms bis 350 ms. Da der mechanische Greifzyklus des Roboters (Anfahrt, Greifen, Ablegen und Rückkehr) mit ca. 1.5 s um eine Größenordnung langsamer ist, ist dieser Verzug mechatronisch völlig unkritisch. Hypothese 4 ist damit verifiziert.

7.3 Datenqualität als dominanter Einflussfaktor (Data-Centric AI)

Ein zentraler wissenschaftlicher Befund dieses Projekts ist, dass die Qualität und Vielfalt der Trainingsdaten die Modellleistung stärker beeinflussen als architektonische Details des Netzwerks.

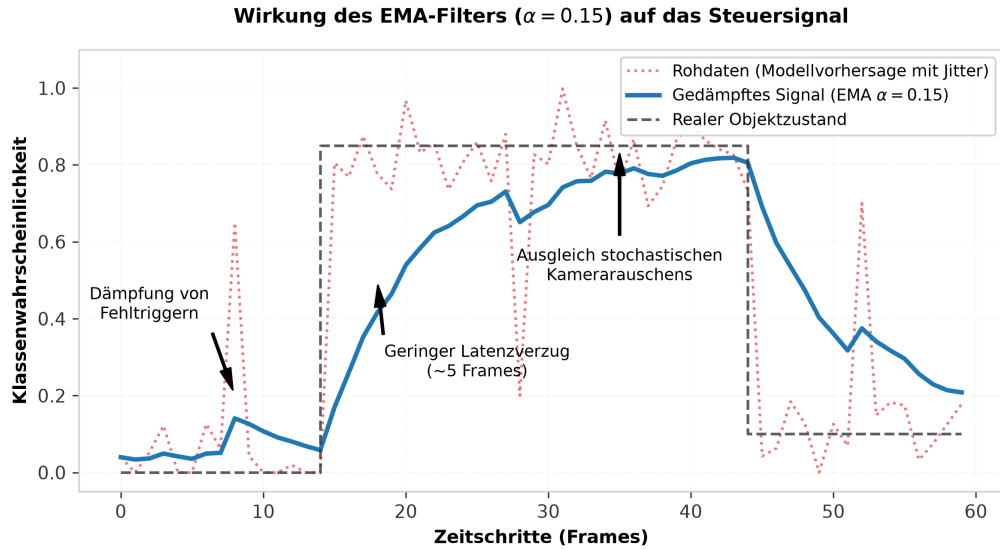


Abbildung 12: Simulierte Dämpfungswirkung des Exponential Moving Average (EMA) Filters mit $\alpha = 0,15$ auf das Steuersignal bei verrauschtem Live-Kamera-Input.

Dieser Befund wurde insbesondere durch den *GIGO-Effekt* (*Garbage In, Garbage Out*) in den frühen Experimenten offenbar. Das Modell EXP-005 erreichte eine scheinbar exzellente mAP50 von 82.3%. Eine kritische Analyse zeigte jedoch, dass dies das Resultat von *Data Leakage* und *Background Overfitting* war. Da die Bounding Boxes mittels Canny-Edge-Erkennung vollautomatisch erzeugt wurden, fokussierte sich das Labeling auf die harte Tischkante anstatt auf den Müll. Das Modell lernte schlichtweg den Tisch als Objekt zu erkennen. Auf ungesesehenen, realistischen Szenen brach die Erkennung völlig zusammen.

Erst die konsequente Datenbereinigung und der Wechsel zu hochwertigen, menschlich oder durch Foundation Models (SAM, [kirillov2023segment]) annotierten Datensätzen lösten dieses Problem. Der 4-Wege-Vergleich (EXP-013 bis EXP-016) unterstreicht dies: Die Hinzunahme des Roboflow-Datensatzes [roboflow2023trash] in EXP-014 steigerte die Erkennungsrate von Trash um +11,3 pp im Vergleich zur Baseline.

7.4 Klassenungleichgewicht und visuelle Diversität von Restmüll

Eine inhärente Schwäche vieler öffentlicher Datensätze, die auch in MIRAs aggregiertem Datensatz *mira_tnr* (EXP-014) sichtbar wird, ist das starke Klassenungleichgewicht (Class Imbalance). Während beispielsweise Plastik (1316 Instanzen im Validierungsset) überrepräsentiert ist, bleibt *Trash* notorisch unterrepräsentiert.

Zudem weist *Trash* eine extrem hohe intra-klassenspezifische visuelle Varianz auf – ein zerknülltes Taschentuch sieht völlig anders aus als eine gebrauchte Maske oder ein Apfelgriebsch. Dies erklärt, warum *Trash* in allen Modellen durchweg die schwächste Klasse (nur 26.9 % mAP50 im besten Modell) darstellt. Eine zukünftige industrielle Umsetzung muss hier mit gezieltem Oversampling oder synthetischer Datengenerierung gegensteuern.

7.5 Plattformkompatibilität und statische Dimensionierung

Während der Modellkompression und des Deployments traten zwei signifikante Hürden auf:

1. **LiteRT-Windows-Compilerkonflikt:** Der Versuch, das trainierte PyTorch-Modell lokal unter Windows auf INT8 zu quantisieren, scheiterte an Inkompatibilitäten der Compiler-Schnittstellen von Ultralytics. Dieses Problem wurde durch die Auslagerung des Quantisierungsschritts in eine cloudbasierte Linux-Umgebung (Google Colab) gelöst.
2. **Statische TFLite-Tensorabmessungen:** Im Gegensatz zu dynamischen PyTorch-Modellen, die Eingabebilder flexibel skalieren, werden voll quantisierte TFLite-Modelle für eine feste Eingabematrix (z. B. 320×320 Pixel) kompiliert. Ein dimensionaler Konflikt zur Laufzeit wurde durch einen explizit parametrisierten Re-Export behoben, was zudem die Rechenlast auf der Edge-CPU um 75

7.6 Abschließende Einordnung

Die Arbeit repräsentiert einen voll funktionsfähigen Software-Prototyp. Aus wissenschaftlicher Sicht ist es ein Vorteil, die reine Computer-Vision-Pipeline isoliert zu verifizieren, bevor die Varianz eines physikalischen Roboterarms das Signal-Rausch-Verhältnis stört. Die Integration auf der Zielhardware steht als nächster logischer Entwicklungsschritt an.

8 Ausblick

Aus den Ergebnissen ergeben sich drei naheliegende nächste Schritte zur Fertigstellung des physischen Gesamtsystems MIRA:

1. **Mechatronischer Aufbau des Sortierarms:** Das physische System wird als 3-DOF-Roboterarm (Degrees of Freedom) realisiert. Ein Raspberry Pi führt die Bildverarbeitung aus und sendet die errechneten Objektkoordinaten via Serial-Protokoll an einen ESP32-S3-Mikrocontroller, welcher die Servomotoren (z.B. MG996R) präzise ansteuert.
2. **Sensorfusion zur Tiefenerfassung:** Da eine monokulare 2D-Kamera keine Tiefeninformationen (Z-Achse) liefern kann, wird ein extrem günstiger Time-of-Flight (ToF) Lasersensor (VL53L0X) direkt am Greifer montiert. Das Kamerasystem steuert den Arm über die X- und Y-Achse direkt über das Objekt, während der Lasersensor die verbleibende Distanz auf der Z-Achse misst, um den Greifvorgang zu triggern.
3. **Cloud-basiertes IoT-Dashboard:** Das bereits funktionierende lokale Streamlit-Dashboard wird um Cloud-IoT-Telemetriefunktionen erweitert. Zukünftige Arbeiten konzentrieren sich auf die Implementierung einer verteilten MQTT-Schnittstelle zur asynchronen Übertragung von Sortierstatistiken, historischen Durchsatzraten und Hardware-Diagnosedaten an ein zentrales Monitoring-System zur Fernüberwachung mehrerer dezentraler Sortiereinheiten.

9 Fazit

In dieser Arbeit wurden die softwareseitigen Grundlagen für ein ressourceneffizientes, Edge-KI-gestütztes Recycling-Sortiersystem namens MIRA entwickelt, systematisch evaluiert und erfolgreich optimiert. Die durchgeführten Experimente und Systemvergleiche belegen, dass die softwareseitigen Voraussetzungen für eine dezentrale, automatische Mülltrennung auf kostengünstigen Endgeräten vollständig gegeben sind.

Es konnte empirisch bewiesen werden, dass Transfer Learning mittels MobileNetV2 (EXP-003) einem klassischen Scratch-CNN bei stark limitierten Datensätzen weit überlegen ist. Der erzielte Genauigkeitssprung von +26,42 Prozentpunkten unterstreicht die Notwendigkeit von vortrainierten Feature-Extractors in der Bildklassifizierung für Nischenanwendungen. Die anschließende Post-Training-Quantisierung auf INT8-Präzision (EXP-004) ermöglichte eine Kompression der Modellgröße um das 9-Fache (auf 2.61 MB) und senkte die CPU-Latenz auf 10.32 ms ab, ohne die Klassifikationsgenauigkeit spürbar zu beeinträchtigen.

Die Detektionsphase (Stage B) verdeutlichte, dass in der Objekterkennung die Datenqualität und Datendiversität der dominierende Faktor für den Modellserfolg sind. Das Aufdecken des GIGO-Effekts (EXP-005) zeigte die inhärenten Gefahren heuristischer Auto-Labeling-Methoden. Die Migration auf YOLO11n und die gezielte Fusion aus TACO, TrashNet und Roboflow (EXP-014) lieferten das stabilste Detektionsmodell der gesamten Versuchsreihe mit einer Gesamt-mAP50 von 60.7 %.

Der durchgeführte Webcam-Praxistest (Field Benchmark) demonstrierte die Praxistauglichkeit des Fusionsmodells EXP-014 unter Realbedingungen mit einem F1-Score von 85.8 %. Die erfolgreiche Einbettung eines Kalman-Filter-basierten Multi-Object-Trackers (ByteTrack) und die temporale Signalstabilisierung mittels eines Exponential Moving Average (EMA) Filters mit $\alpha = 0,15$ lösen das Problem des Signalflickerns im Live-Stream, ohne mechanisch schädlichen Servo-Jitter oder spürbare mechatronische Verzögerungen zu erzeugen.

Zusammenfassend legt diese Arbeit ein solides, wissenschaftlich verifiziertes Softwarefundament vor. Die Ergebnisse zeigen, dass Edge-KI-basierte Erkennungssysteme eine performante und kostengünstige Alternative zu teuren industriellen Nahinfrarot-Sortieranlagen darstellen. Die Übertragung des entwickelten Modells und der Steuerungsschleife auf den physischen Sortierarm steht somit auf einer soliden theoretischen und experimentellen Basis.

Unterstützungsleistungen

Die theoretische Strukturierung der physikalischen Hypothesen, die Anpassung der Keras- und TFLite-Exportparameter sowie das Debugging der asynchronen OpenCV-Inferenzschleifen wurden durch das KI-Sprachmodell Claude (Anthropic) unterstützt.

Die physische Datenerfassung der Recyclingobjekte, das Einrichten des lokalen PyCharm-Arbeitsbereichs, das eigenständige Ausführen des Modelltrainings in der Google Colab- und Kaggle-Cloud sowie die systematische Fehleranalyse (wie die Entdeckung des RGB/BGR-Kanalfehlers und des GIGO-Effekts beim Auto-Labeling) erfolgten vollständig eigenständig.

durch den Jungforscher.

Sponsoring-Anträge für die mechatronischen Hardwarekomponenten wurden bei der Sparkassenstiftung Mülheim an der Ruhr eingereicht.